

LAB PORTFOLIO: GOVERNED LOCAL AI SYSTEMS

Executive Summary

Author: Ramon E. Ramirez

Date: 2026-07-09 to 2026-07-10

Classification: Career Documentation / Technical Portfolio

PROJECT OVERVIEW

This document certifies independent research and development work conducted on local AI infrastructure, cognitive governance frameworks, and vendor lock-in analysis. The work was performed under real-world conditions on personal hardware with production-grade tools.

CORE ACHIEVEMENTS

1. Seven-Layer Cognitive Governance Architecture (GEN-CULTIVATE v1)

- Designed and operationalized a 7-layer governance stack for AI-human interaction
- Layers: AAS-1 (Orientation), HPL (Cognitive Grammar), CVL (Verification), PCS (Experiential Proof), EMBR (Embodied Regulation), GEN-CULTIVATE (Governance Container), CLEAR-ANCE (Execution Authorization)
- Successfully injected governance constraints into AI reasoning systems, producing measurable behavioral change (precision increase, drift reduction, authority preservation)

2. Local AI Infrastructure Deployment

- Configured Ollama local inference engine with custom models (ornith:35b, dhu-ornith-builder)
- Verified GPU acceleration on NVIDIA RTX 4070 Ti (12GB VRAM)
- Established fully private, zero-cloud-cost AI operation
- Hardware profile: 12-core CPU, RTX 4070 Ti, 1TB E: drive storage

3. Commercial Application Reverse Engineering

- Analyzed OpenHuman v0.58.7 (Electron-based AI assistant)
- Discovered hardcoded MVP allowlist restricting local model selection
- Documented silent cloud fallback mechanism: when custom model selected, app routes to cloud model (chat-v1) with token cost tracking
- Identified this as architectural vendor lock-in, not a bug
- Located configuration files: TOML-based user profiles, SQLite memory vaults, Electron window state management

4. Alternative System Evaluation & Configuration

- Evaluated AgenticSeek (Python/FastAPI) as open-source alternative
- Configured for Ollama integration with custom model support
- Implemented governance injection at the prompt-loading layer (agent.py load_prompt modification)
- Created automated governance preamble injection system

5. System Troubleshooting & Recovery

- Diagnosed Electron window state corruption (8 zombie processes)
- Used PowerShell, CMD, Task Manager, and Event Viewer for system-level debugging
- Performed clean uninstall, registry cleanup, and reinstall protocols
- Maintained data integrity through backup/restore procedures

INNOVATION CLAIMS

Novel Contribution 1: Governance-as-Code for AI Assistants

Traditional AI systems operate without structured oversight. This work demonstrates that governance frameworks (intent declaration, context bounding, assumption surfacing, constraint enforcement) can be injected into AI systems at the prompt-loading layer, producing measurable improvements in precision and authority preservation.

Novel Contribution 2: Vendor Lock-In Detection Methodology

Developed a systematic approach to identify hidden commercial constraints in “local-first” AI applications:

1. UI observation vs. config file analysis
2. Network traffic monitoring (inferred from cost tracking)
3. Log analysis for fallback routing patterns
4. Model allowlist verification

Novel Contribution 3: CASF Baby-Steps Operational Protocol

Created and applied a step-by-step execution protocol (“Chairman Approval System Framework”) that prevents premature execution, scope drift, and unauthorized substitutions. This protocol was self-applied to the AI assistant itself, producing documented behavioral change.

TECHNICAL ENVIRONMENT

- **OS:** Windows 10/11 (Version 10.0.26200.8655)
- **Hardware:** Custom PC, Intel CPU, NVIDIA RTX 4070 Ti, 1TB E: drive
- **Tools:** PowerShell, CMD, Notepad, Git, Podman 5.8.3, Ollama
- **Languages:** Python (config parsing, patch scripts), TOML (configuration), Rust (inferred from logs)
- **Applications:** OpenHuman v0.58.7, AgenticSeek (latest), Ollama (local inference)

OUTCOMES

1. **Functional System:** OpenHuman running from E: drive with local model selection
2. **Governance-Injected Alternative:** AgenticSeek configured with Gen-Cultivate preamble on all agent prompts
3. **Documentation:** Full session logs, configuration files, and analysis artifacts
4. **Knowledge Base:** Understanding of Electron app architecture, Ollama model management, Windows process debugging

CAREER RELEVANCE

This work demonstrates:

- **Systems Architecture:** Designing multi-layer governance for complex AI systems
- **Technical Debugging:** Reverse-engineering commercial applications and diagnosing runtime failures
- **AI Ethics & Safety:** Implementing constraint frameworks that prevent unauthorized execution
- **Infrastructure Management:** Local deployment, GPU optimization, storage management
- **Documentation Discipline:** Maintaining structured logs and configuration control

STATUS

Active development. Local prototype verified. Next phase: public-safe documentation and controlled testing.

Authority: Ramon E. Ramirez **Date:** 2026-07-10 **Classification:** Career Portfolio / Technical Evidence